Game Server & Engine Programmer Portpolio

Autor : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738







PIP Project

프로젝트 개요

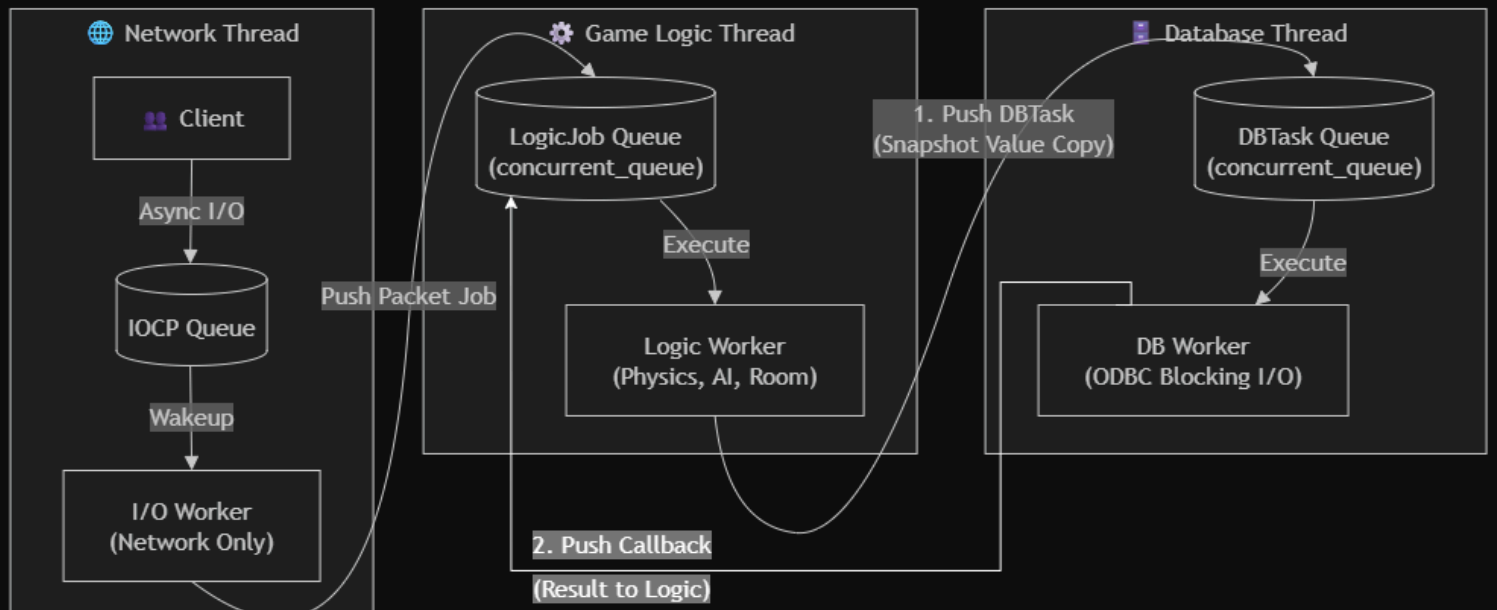




프로젝트 이름	**PIP Project [S.T.L(Slay The Lord):구원자의 뒤틀린 웃음]**
장르	Action RPG (DirectX 12 + Jolt Physics)
설명	MO 액션 RPG
담당 역할	Server Physics & AI Sync, Client Engine Architecture
AI 활용	gemini pro 3.1, Antigravity를 자료조사 및 수정 계획 생성 및 코드 생성에 사용 검증은 직접
기술 스택	**C++23** **Jolt Physics** **Behavior Tree** **IOCP** **DX12** **Win32**
개발기간	2025/08/01 ~ 2026/06/22
개발 인원	3인

PIP Server Architecture

서버 아키텍쳐 전체적인 설계도입니다.



Behavior Tree(BT)를 활용한 보스 AI

Problem: 보스의 복잡한 페이즈(Phase) 변화, 스킬, 타겟팅 로직이 얽혀 스파게티 코드 가능성

Solution: 행동 트리(BT)의 Action 노드를 상속 구조로 모듈화하고, 타겟(유저) 상실 시 즉시 대응하는 예외처리(안전장치) 노드를 Decorator로 배치

Performance Gain: 보스 패턴 로직의 재사용성 확보

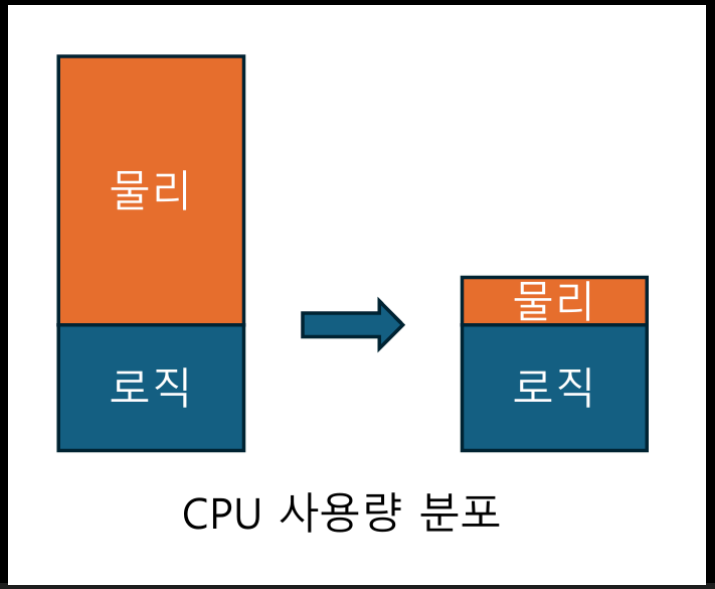
```
BTBuilder builder;
auto root = builder
    .sequence()
    .leaf<Condition_IsAlive>()
    .leaf<Condition_IsHitted>(DecoratorType::Inverter)
    .selector() // 전체 로직 우선순위 결정
        // --- [우선순위 1] 전투 로직 ---
        .sequence()
            .leaf<Condition_HasEnemy>() // 타겟(적군)이 있는가?
            .leaf<Condition_IsTargetInLeashRange>(25.0f) // 25m 내에 있으면 추격 유지
            .selector()
            // 1-1. 강력한 공격 시도 (사거리 2.5m)
                .sequence()
                    .leaf<Condition_IsEnemyInRange>(1.5f)
                    .leaf<Action_AttackEnemy>(heavyAtk)
                .end()
            // 1-2. 일반 공격 시도 (사거리 1.5m)
            .sequence()
                .leaf<Condition_IsEnemyInRange>(1.5f)
                .leaf<Action_AttackEnemy>(normalAtk)
            .end()
            // 1-3. 타겟 사거리 밖이면 네비메쉬 추격
            .sequence()
                .leaf<Action_UpdateEnemyPosToTarget>()
                .leaf<Action_FindPath>("MainStage_NavMesh")
                .leaf<Action_FollowPath>(6.0f) // 추격 속도 6.0
```

서버 물리 엔진(Jolt) 부하 억제 설계

Problem: 다수의 플레이어와 몬스터 객체 물리 연산을 서버가 모두 담당할 경우 심각한 물리 스레드 병목 발생

Solution: 공간 그리드로 플레이어 주변 셀의 NPC만 매 틱 활성 목록(_activeNpcList)에 포함시켜 갱신 대상 자체를 축소. 보스는 Jolt CharacterVirtual 정밀 물리를 그대로 쓰고, 일반 NPC는 레이캐스트 1회로 끝나는 경량 LightPhysicsUpdate로 대체 (정지+접지 상태면 그마저 스킵)

Performance Gain: 물리 갱신 대상과 연산량을 동시에 줄여 안정적 서버 물리 프레임 타임 확보 (전체 CPU 사용량 84% -> 11%로 감소)




```
// Room::UpdatePhysics — 전체 NPC 대신 시야 내 활성 NPC만 순회
for (auto* npc : _activeNpcList) {
    if (npc->is_boss()) { bosss.push_back(npc); continue; }
    auto nc = npc->GetNPCController();
    if (!nc || !npc->IsActive()) continue;
    nc->LightPhysicsUpdate(deltaTime);   // 일반 NPC는 경량 갱신
}
for (auto& boss : bosss)
    boss->PhysicsUpdate(deltaTime, tempAllocator);  // 보스만 정밀 물리

// NPCControllerComponent::LightPhysicsUpdate
bool isMovingXZ = (LengthSq(_aiVelocity) > 0.0001f ||
                   LengthSq(_impactVelocity) > 0.0001f);
if (!isMovingXZ && _verticalVelocity > -0.2f) return;  // 정지+접지 시 스킵
// CastShape(무거움) 대신 CastRay(가벼움)로 대체
```

정밀 피격 동기화를 위한 지연 보상 (Rewind)

Problem: 네트워크 지연(Ping)으로 인해 클라이언트 화면의 몬스터 위치와 서버 위치가 달라 타격감 훼손

Solution: 각 액터의 최근 1초 위치/회전을 deque 히스토리로 기록. 공격 패킷의 클라이언트 타임스탬프를 세션별 관측 최소 지연(clock offset) 대비 초과분만큼만 되감아(최대 1초 clamp) 그 시점의 히트박스 트랜스폼을 재구성한 뒤 Jolt로 충돌 판정

Performance Gain: 클라이언트-서버 시계 오프셋은 자동 보정하고 핑 변동분만 되감아, 지연 환경에서도 실제 타격 시점 기준으로 판정 불일치 최소화



```
// Actor::RecordSnapshot / GetSnapshotAt — 1초 위치·회전 히스토리
void RecordSnapshot(uint32_t timestamp) {
    _history.push_back({ timestamp, GetPosition(), GetRotation() });
    if (_history.size() > 30) _history.pop_front();  // 1초 유지
}
ObjectSnapshot GetSnapshotAt(uint32_t timestamp) const {
    auto it = std::lower_bound(_history.begin(), _history.end(),
timestamp,
        [](auto& s, uint32_t t) { return s._timestamp < t; });
    return (it == _history.end()) ? _history.back() : *it;
}

// Player::ComputeRewindTimestamp — 클록 오프셋 자동 보정 + 초과 지연만 되감
기
uint32_t rawDelta = serverNow - clientTimeStamp;
if (!_actionClockOffsetInit || rawDelta < _actionClockOffsetMs)
    _actionClockOffsetMs = rawDelta;   // 관측된 최솟값을 기준선으로
uint32_t rewindMs = std::min(rawDelta - _actionClockOffsetMs,
MAX_REWIND_MS);
return serverNow - rewindMs;

// HitboxComponent::CheckCollision — 되감은 과거 트랜스폼으로 충돌 판정
JPH::RMat44 npcWorld = JPH::RMat44::sRotationTranslation(
    ToJolt(pastData._rotation), ToJolt(pastData._position));
JPH::CollisionDispatch::sCollideShapeVsShape(attackShape, hb.shape, ...,
attackTransform, npcWorld * hbLocal, ...);
```

Data Race가 없는 안전한 Room(인스턴스 던전) 아키텍처

Problem: 한 방(Room)에 속한 객체 상태를 여러 워커 스레드가 동시 다발적으로 접근하여 수정하면 데이터 오염 및 데드락 위험

Solution: 방(Room)마다 고유한 Job Queue(명령 대기열)를 부여하여, 상태를 변경하는 모든 로직(이벤트)을 큐에 밀어넣고 단일 스레드 흐름으로 순차 처리(직렬화)

Performance Gain: 객체 단위의 동기화 락(Lock) 비용 완전 제거로 인해 서버 반응 속도 향상 및 데드락 100% 방지, MO RPG 게임 특성 활용



성과: 2026 한국게임학회 춘계 학술대회 논문 포스터 발표



한국공학대학교 TECH UNIVERSITY OF KOREA 2026 한국게임학회 춘계 학술발표대회 한국게임학회 Korea Game Society

3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스키닝 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

한국공학대학교 박대원, 임찬진, 박경준, 정내훈

Introduction

외부 라이브러리 없는 독자적 glTF 2.0 파서 개발 및 스키닝 애니메이션 시스템 구현
DirectX 12 기반 실시간 PBR 파이프라인 구축으로 시각적 사실성 및 렌더링 성능 확보
IOCP 서버와 Jolt Physics 엔진 연동을 통한 3D MO 게임 맞춤형 물리 시뮬레이션 환경 완성
스냅샷 리와인드 지연 보상 및 물리 LOD 적용으로 다중 접속 환경의 물리 연산 병목 해소

glTF 2.0 파싱 및 스키닝 애니메이션 시스템

- 외부 라이브러리 없이 바이너리를 직접 1차원 배열로 파싱해 캐시 효율을 높이고, 스케일 행렬을 곱해 왼손 좌표계 이기종 문제를 해결
- 이진 탐색을 적용해 키프레임 탐색 시간을 단축하고, 마스터 스켈레톤 기반의 소켓 시스템으로 무기 장착을 구현
- 선형 할당기를 설계하여 렌더링 시 발생하는 할당 오버헤드와 단편화를 차단
- 마스터 OBB 기반의 계층적 BVH 구조를 통해, 메쉬와 카메라 간의 충돌 연산 부하를 대폭 감소

Jolt Physics 서버 연동 및 물리 엔진 최적화

- Room 인스턴스 단위로 독립적인 물리 공간을 격리하고, 계층적 충돌 필터링을 통해 Broad-phase 단계의 불필요한 충돌 쌍을 차단함.
- 정적 맵 데이터(StaticCompoundShape)를 단일 생성 후 인스턴싱하여 메모리를 최적화함.
- 거리 기반 물리 LOD를 적용하여, 원거리 NPC는 RayCast 제어로 전환하고 메인 루프 호출을 제한함.

- Transform, CharacterController, NPCController 등을 계층적 컴포넌트 구조로 분리하여 로직 제어 효율성을 높임.
- 부위별 Hitbox 데이터와 스냅샷 리와인드(Snapshot Rewind)를 결합하여, 공격 패킷 수신 시 과거 위치를 역산해 지연을 보상함.

실시간 PBR 파이프라인 및 IBL 통합

- ORM 텍스처를 분리하여 차폐(R), 거칠기(G), 금속성(B) 파라미터로 해석해 메모리 대역폭을 절약
- Cook-Torrance BRDF 모델을 적용하여 직접광 반사를 구현
- Diffuse IBL은 9개의 SH 계수를 통해 연산량을 감축
- Specular IBL은 Split-Sum Approximation을 도입해 LUT를 조합하여 실시간 렌
- Fresnel 근사식을 적용하
  금속/비금속 재질의 물리적 반사 특성 구현
- 표준 샘플 모델 기반의 시각적 검증을 통해 PBR 파이프라인의 렌더링 신뢰성 입증

Result

- 독자적 glTF 파싱으로 메모리 캐시 효율을 극대화하고 좌표계를 보정. 이진 탐색을 통해 애니메이션 연산을 O(log N)으로 단축함과 동시에 마스터 스켈레톤 소켓 시스템을 구현하였고, 선형 할당기 제작을 통해 렌더링 시의 메모리 단편화를 차단하였으며, 마스터 OBB 기반 계층적 BVH를 도입해 실시간 충돌 연산 부하를 대폭 감소
- Cook-Torrance BRDF와 향상된 IBL 기법(SH 계수, Split-Sum 근사)을 적용한 최적화된 PBR 파이프라인을 구축하고 glTF 2.0 파서를 결합해 연산 부하를 절감함으로써, 상용 엔진 없이도 수준의 시각적 사실성과 실시간 렌더링 성능을 입증
- 서버 권위적 판정, 스냅샷 기반 지연 보상시스템을 결합하여, 클라이언트 변조를 원천 차단하고 네트워크 지연 상황에서도 높은 타격 신뢰성을 보장이 가능하며, 400명 플레이어 + 5만마리 NPC 동시 물리시뮬레이션 가능한 아키텍쳐 구현



MO게임서버 jolt 오픈소스 물리엔진 연동과 물리엔진 부하 최적화, 스냅샷 리와인드 연구를 진행하였고 2026 한국게임학회 춘계 학술대회 논문 포스터 발표를 하였습니다.

<-논문 포스터

DB 스레드 분리 및 Stored Procedure 설계

Problem: 멀티플레이 환경에서 인벤토리를 영속화해야 했지만, ODBC 표준 API는 동기 블로킹 방식. 워커 스레드는 방 하나의 플레이어·몬스터·Jolt 물리 Step을 통째로 굴리기 때문에, 쿼리 한 번이 수십 ms를 잡아먹으면 안됨. 그렇다고 DB 스레드에 컴포넌트 포인터를 그냥 넘기면 데이터 레이스 발생

Solution: DB 전담 1개 신설. 작업 단위인 DBTask는 페이로드를 std::any로 담아 타입을 소거 — 저장은 로직 스레드가 인벤토리를 값 복사한 스냅샷을 make_any로 이동시켜 넣고, 로드는 반대로. 요청 시점의 logic_thread_idx를 사용, DB 스레드는 완료 콜백을 그 인덱스의 로직 큐로만 되돌려주고 로직 워커 스레드에서 싱글 스레드 알고리즘으로 처리

좋은 점: 로직 스레드는 즉시 리턴하므로 DB 지연이 게임 틱에 영향 없음, 완료 콜백이원래 소유 스레드에서 실행되므로 Player·InventoryComponent 접근에 Mutex가 하나도 필요 없습니다. std::any 덕분에 새 DB 작업(스탯 등)을 추가해도 DBTask 구조체와 큐는 그대로 두고 핸들러만 등록하면 되며, 연결 문자열을 "DUMMY"로 주면 SQL만 건너뛰고 큐·콜백 경로는 동일하게 타므로 팀원이 MSSQL 설치 없이 같은 비동기 코드 경로를 테스트할 수 있습니다.

```cpp
// — 1) 스레드 구성 : IOCP I/O 2 + 로직(P코어-2) + DB 전담 1 ————————
auto p_cores = GetPerformanceCoreIndices();          // 하이브리드 CPU에서 P코어만 선별
int logic_count = std::max(1, (int)p_cores.size() - 2);
Server::Instance()->Start(/*io*/2, logic_count);      // DBManager는 내부에서 DB 스레드 1개 생성

// — 2) 작업 단위 : 페이로드를 std::any로 타입 소거 ————————————————
struct DBTask {
    DBTaskType    type;
    int           logic_thread_idx;  // ★ 콜백을 돌려받을 스레드 = 소유권 티켓
    DBJobCallback callback;          //   로직 스레드에서 실행될 함수
    std::any      data;              // ★ 타입 소거된 페이로드
};

// — 3) 저장 : 값 복사 스냅샷을 std::any로 이동 (공유 메모리 제거) ————
InventorySnapshot snap;
snap.materials = inven->get_materials_snapshot();     // NRVO 값 복사
task.logic_thread_idx = _logic_thread_idx;
task.data = std::make_any<InventorySnapshot>(std::move(snap));
DBManager::Instance()->push_task(std::move(task));    // 논블로킹, 즉시 리턴

// — 4) 반환 : 완료 콜백을 '요청한 그 스레드'로만 push ————————————
auto* q = Server::Instance()->get_logic_queue(task.logic_thread_idx);
q->push({ std::move(task.callback) });                // → any_cast 후 Mutex 없이 갱신
```



SImplest Minecraft MMO RPG GAME

프로젝트 개요





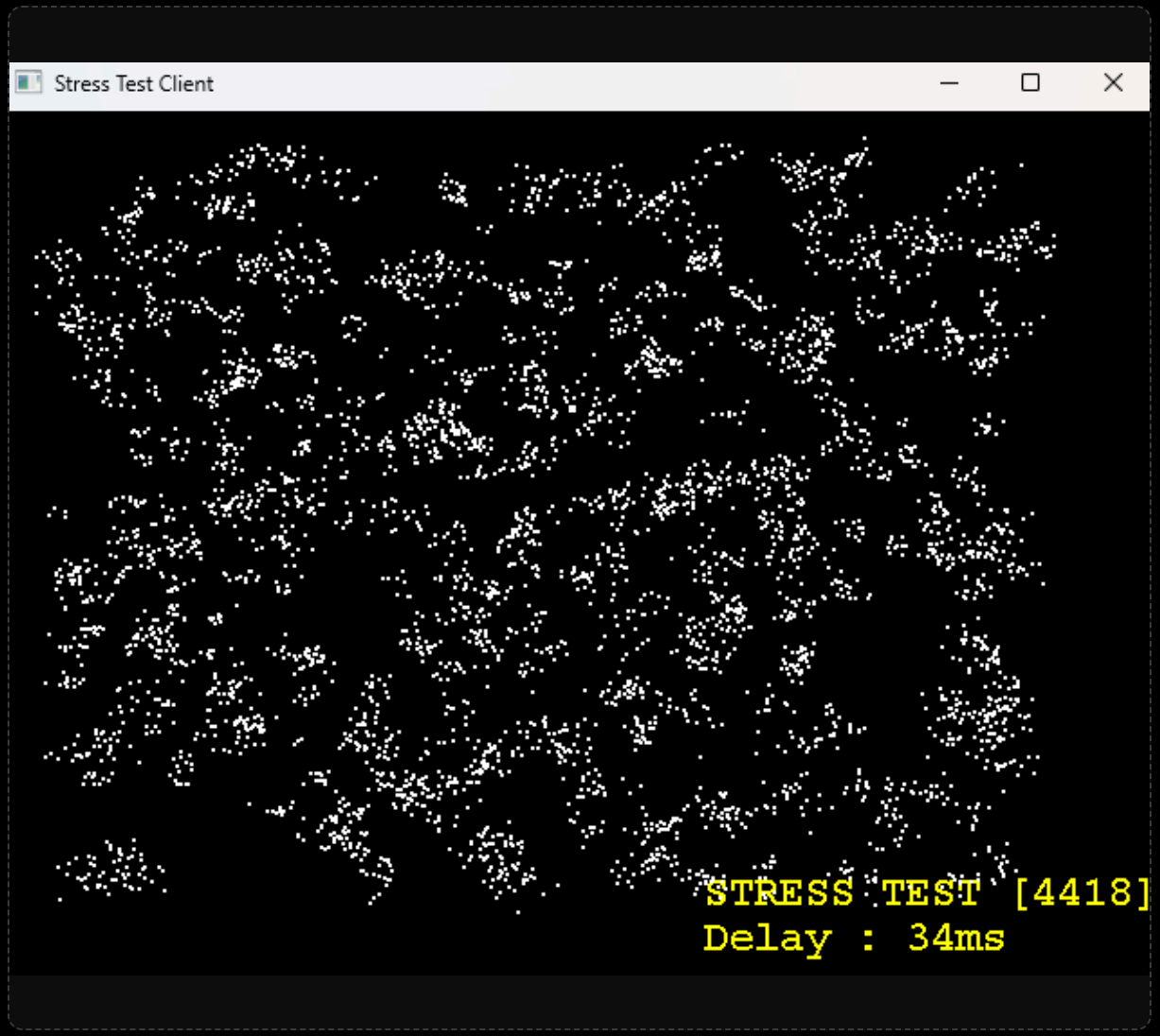
프로젝트 이름	SImplest Minecraft MMO RPG GAME
장르	MMORPG
설명	IOCP 기반 C++ 멀티스레드 서버. 동접 4400명 및 단일 맵, 20만6개 객체 동시 구동
담당 역할	Server Architecture & Optimization, Client Devlop
AI 활용	Antigravity(gemini pro)사용 개발중 코드 생성에 사용
기술 스택	C++ IOCP ODBC MultiThreading Lua
개발기간	2026/05/04 ~ 2026/06/22
개발인원	1인

시야 처리 병목 최적화 (Sector Grid)

Problem: 20만 마리의 NPC와 유저가 서로의 거리를 O(N^2)로 계산하여 서버 틱 심각한 저하 발생

Solution: 공간을 고정 크기의 Grid(Sector)로 분할하고 std::shared_mutex(Read-Write Lock)를 적용하여 인접 섹터의 객체만 탐색하도록 변경

Performance Gain: 탐색 복잡도를 O(k) 수준으로 낮추어 동시 접속자 수 4400명 안정적 유지



```
struct cell { // Sector — 맵을 SECTOR_SIZE(10) 단위 격자로 나눠 관리하는 공간 분할 구조
    std::unordered_set<int> objects;
    std::shared_mutex mtx;   // 섹터별 독립 락 — 다른 섹터끼리는 락 경합 없음
};
class Sector {
    std::array<std::array<cell, WORLD_WIDTH / SECTOR_SIZE>,
                           WORLD_HEIGHT / SECTOR_SIZE> object_sector;

    // 오브젝트가 실제로 다른 셀로 넘어갈 때만 갱신 (같은 셀 내 이동은 스킵)
    void move_object(int id, int old_x, int old_y, int new_x, int new_y) {
        int osx = old_x / SECTOR_SIZE, osy = old_y / SECTOR_SIZE;
        int nsx = new_x / SECTOR_SIZE, nsy = new_y / SECTOR_SIZE;
        if (osx == nsx && osy == nsy) return;

        { std::unique_lock lock(object_sector[osy][osx].mtx);
          object_sector[osy][osx].objects.erase(id); }
        { std::unique_lock lock(object_sector[nsy][nsx].mtx);
          object_sector[nsy][nsx].objects.insert(id); }
    }

    // 시야(range) 범위가 걸치는 섹터들만 순회 — 전체 스캔 대신 O(주변 셀 수)
    std::vector<int> get_objects_nearby_sector(int x, int y, int range = VIEW_RANGE) {
        std::vector<int> result;
        int sx0 = std::max(0, x-range)/SECTOR_SIZE, sx1 = std::min(WORLD_WIDTH-1,
x+range)/SECTOR_SIZE;
        int sy0 = std::max(0, y-range)/SECTOR_SIZE, sy1 = std::min(WORLD_HEIGHT-1,
y+range)/SECTOR_SIZE;

        for (int sy = sy0; sy <= sy1; ++sy)
            for (int sx = sx0; sx <= sx1; ++sx) {
                std::shared_lock lock(object_sector[sy][sx].mtx);  // 읽기는 동시 허용
                result.insert(result.end(), object_sector[sy][sx].objects.begin(),
                                            object_sector[sy][sx].objects.end());
            }
        return result;
    }
};
```

네트워크 I/O 병목 제거: WSASend 배칭 & 링 버퍼 기반 재조립

Problem (원인): 매 recv마다 패킷 경계가 TCP 버퍼 끝과 어긋나면(단편화) 남은 조각을 버퍼 앞으로 memmove로 옮겨야 함 — 남은 데이터가 클수록 비용이 커지는 O(n) 연산이고, 그 다음 recv도 이 위치를 기준으로 다시 계산해야 함

Solution (내용): 선형 버퍼를 원형 큐(RingBuffer)로 교체 — head/tail 포인터 이동만으로 랩어라운드(wrap-around)를 처리하고, 완성된 패킷이 있을 때만 그 크기만큼만 복사

Performance Gain (결과):포인터 연산 기반 O(1) 처리로 부하가 늘어도 비용이 커지지 않음 — 실측 패킷당 평균 109ns (동접 1만+, 초당 약 6,700회 호출, 안정적으로 수렴). 단편화가 잦은 실제 서비스망 조건에서 구조적 이점이 커지는 개선

```cpp
// RingBuffer.h — Peek: head/tail 포인터로 랩어라운드 자동 처리, 필요한 만큼만 복사
bool Peek(char* dest, int size) {
    if (GetStoredSize() < size) return false;
    int directReadSize = GetDirectReadSize();
    if (size <= directReadSize) {
        std::memcpy(dest, &_buffer[_head], size);
    } else {
        std::memcpy(dest, &_buffer[_head], directReadSize);
        std::memcpy(dest + directReadSize, &_buffer[0], size - directReadSize);
    }
    return true;
}

// worker_thread — Before: 남은 데이터를 버퍼 앞으로 통째로 이동 (O(n))
if (data_size > 0) {
    memmove(cl->recv_over_.buff, p, data_size);
    cl->prev_recv_count_ = data_size;
}

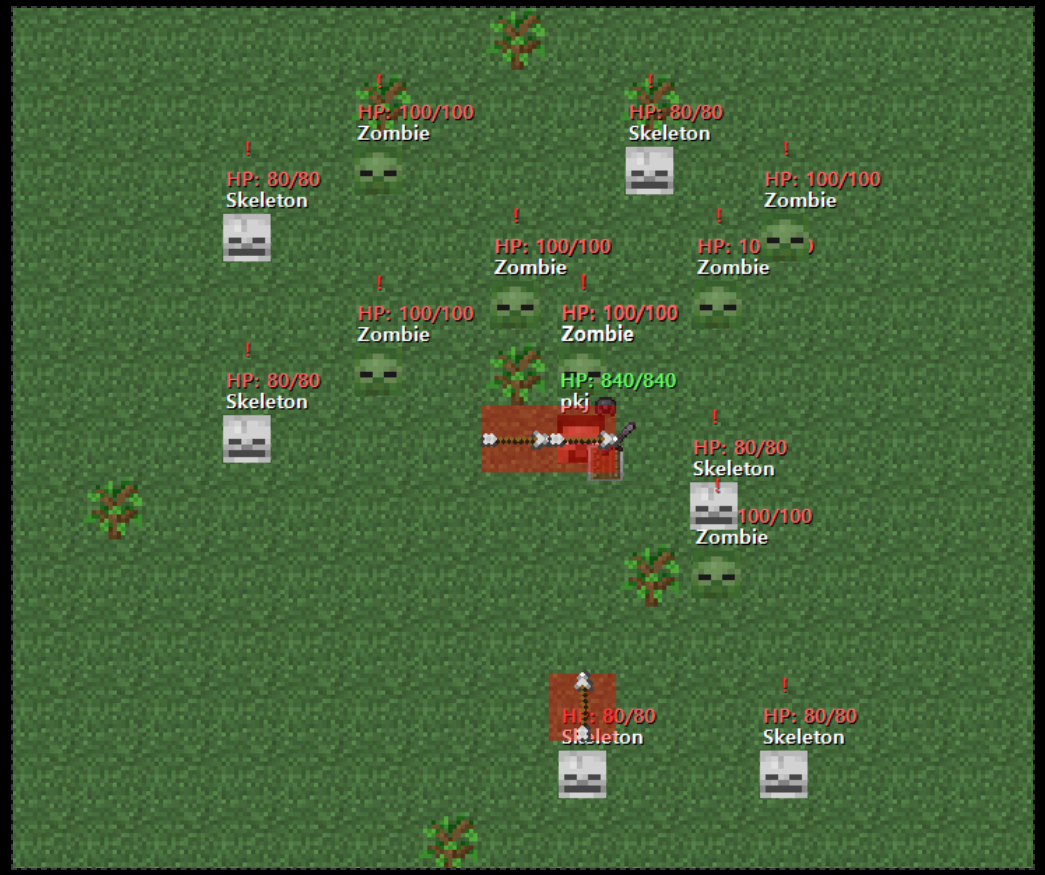
// worker_thread — After: 포인터만 전진, 완성된 패킷만 필요한 만큼 복사 (O(1))
cl->recv_buffer_.CommitWrite(num_bytes);
while (true) {
    unsigned char packet_size;
    if (!cl->recv_buffer_.Peek((char*)&packet_size, 1)) break;
    if (cl->recv_buffer_.GetStoredSize() < packet_size) break;
    cl->recv_buffer_.Read((char*)packet_buf, packet_size);
    cl->proccess_packet(packet_buf);
}
```

LUA Script를 활용한 확장성 있는 NPC AI

Problem: 20만 객체의 행동 패턴을 C++에 하드코딩하면 밸런스 한 줄을 고칠 때마다 재컴파일·재배포가 필요합니다. 그렇다고 Lua를 붙이면 lua_State가 재진입 불가라, 전역 하나를 공유하는 순간 모든 워커 스레드가 AI 호출마다 뮤텍스를 두고 경쟁해서 성능이 떨어집니다.

Solution: thread_local lua_State 풀을 두어 워커 스레드마다 독립적인 VM 7개를 보유하게 했습니다. 이를 가능하게 한 핵심 설계는 AI 스크립트를 순수 함수로 강제한 것입니다 — NPC의 모든 상태(HP·좌표·상태·타겟)는 C++가 소유하고 매 호출 ctx 테이블로 주입, 결과는 new_state·action으로만 반환받습니다. 보스 페이즈 전환과 크리퍼 자폭 카운트다운까지 이 왕복으로 처리해, lua_State를 상태 저장소가 아닌 순수 실행 엔진으로 만들었습니다. (저장,실행은 C++, 판단만 lua)

Result: AI 호출 시 락 획득 0회, lua_State 개수 = 스크립트 7종 × 코어 수 (16코어 기준 112개) — NPC가 20만이든 100만이든 불변 · NPC 이벤트를 IOCP로 아무 워커나 처리 — 스레드 핀닝·라우팅 로직 불필요, 부하 분산이 자동 · 한 스레드의 Lua 에러가 lua_pcall에 격리되어 다른 스레드로 전파되지 않음 · GC가 State 단위로 분산되어 전역 방식의 stop-the-world 정지가 없음 · 밸런스 수치는 npc_config.lua, 행동은 ai/*.lua로 분리



Lua Script를 활용한 효율적인 NPC AI

1. 구현 구조

상태 저장소가 두 개로 갈려 있습니다.

```
// npc_ai_manager.h
// (A) 전역 1개 — 경로만 보관
static std::unordered_map<int, std::string>& script_paths() {
    static std::unordered_map<int, std::string> paths;   return paths;
}
// (B) 스레드마다 1개 — 실제 VM 보관
static std::unordered_map<int, lua_State*>& get_pool() {
    thread_local std::unordered_map<int, lua_State*> pool;   return pool;
}
```

```
// main() — 경로만 등록 (실제 로드는 각 스레드가)
const std::string ai_dir = "ai/";
NpcAiManager::register_script(1, ai_dir + "zombie_ai.lua");
NpcAiManager::register_script(2, ai_dir + "skeleton_ai.lua");
// ... creeper, enderman, iron_golem, ender_dragon, fire_zone

// worker_thread() 진입부 — 이 스레드 전용 VM 7개 생성
NpcAiManager::init_thread_states();
```

```
static void init_thread_states() {
    auto& pool = get_pool();
    for (auto& [type_id, path] : script_paths()) {
        if (pool.contains(type_id)) continue;
        lua_State* L = luaL_newstate();
        luaL_openlibs(L);
        if (luaL_dofile(L, path.c_str()) != LUA_OK) { /* 로그 후 lua_close */ continue; }
        pool[type_id] = L;
    }
}
```

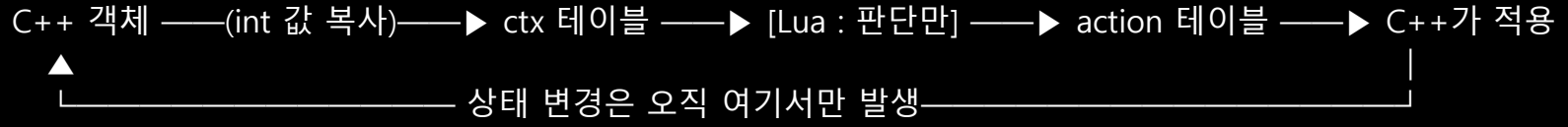
호출 경로:

```
NPC 타이머 만료 → PostQueuedCompletionStatus(h_iocp, -1, obj_id, &move_over->over)
                → 아무 워커 스레드가 수신 (io_type::npc_move)
                → NpcAiManager::call_on_timer(npcType, ctx)
                → call_hook: lua_getglobal → ctx 테이블 push → lua_pcall → 반환 테이블 파싱
                → NpcAction (MOVE / MELEE / AOE / BREATH / DIE ...) 을 C++가 실행
```

혹은 3개: `on_timer(ctx)`, `on_hit(ctx, attacker_id, damage)`, `on_spawn(ctx)`.

이 설계가 가능한 조건 - 모든 LUA AI는 읽기만 쓰기는 C++만

골렘이 순찰하려면 "내가 원래 어디서 태어났는지"를 알아야 하는데,
그 initial_x/initial_y마저 Lua에 저장하지 않고 C++가 들고 있다가 매 호출 ctx에 실어 보냅니다.
"기억이 필요해 보이는" 것 — 보스 페이즈, 자폭 카운트다운, 순찰 기준점



이점:
- NPC가 워커 스레드를 옮겨 다녀도 무결 (상태가 C++에 있으므로)
- 세이브/로드가 자연스러움 — NPC 상태가 전부 C++ 구조체라 그대로 직렬화하면 끝.
- Lua VM을 덤프할 필요가 없음
- AI 단위 테스트가 쉬움 — ctx만 만들어 넣으면 되는 순수 함수라 서버 없이 검증 가능

⚠ 단 하나의 예외 — `math.random`의 RNG 상태

엄밀히 말하면 `lua_State` 안에 가변 상태가 하나 남아 있습니다.

```lua
local d = dirs[math.random(1, 4)]           -- iron_golem_ai.lua : 순찰 방향
if math.random(1, 100) <= 60 then ... end   -- ender_dragon_ai.lua : 패턴 선택
```

DataBase 서버 연동

Problem: 멀티플레이 게임에서 유저의 로그인 정보와 진행 상황(좌표·레벨·장비·인벤토리·퀘스트·스킬)을 영속화할 저장 시스템이 필요했습니다. 그런데 ODBC 표준 API는 동기 블로킹이라, IOCP 워커 스레드에서 직접 호출하면 쿼리가 끝날 때까지 그 스레드가 통째로 묶여 다른 유저의 패킷 처리까지 정지합니다. 로그인 한 번에 조회가 5번 필요한 구조라 접속이 몰리면 워커 풀이 전부 DB 대기로 빠지는 기아 상태가 발생합니다.

Solution: Windows가 ODBC 표준 API를 지원한다는 점을 활용해 MSSQL + Management Studio로 스키마와 저장 프로시저를 직접 설계하고, ODBC 핸들을 DB 전용 접근 스레드에 배치했습니다. 워커 스레드는 DBTask를 lock-free 큐에 던지고 즉시 복귀하며, 페이로드는 std::variant<monostate, string, player_data>로 담아 요청과 응답이 같은 객체로 왕복하게 했습니다. 완료 통지는 별도 채널을 만들지 않고 PostQueuedCompletionStatus로 IOCP에 되돌려, DB 결과가 일반 네트워크 I/O 완료와 동일한 경로로 배달되게 했습니다.

좋은 점: 메인 로직·I/O 스레드와 별개의 스레드에서 비동기로 연산이 처리되므로 DB가 아무리 느려도 게임 로직은 계속 돌아갑니다. 완료 통지에 IOCP를 재사용한 덕분에 콜백 큐·디스패처 없이 기존 io_type switch에 case 하나만 추가하면 됐습니다. 또한 모든 쿼리를 저장 프로시저로 캡슐화해 C++ 코드에 테이블·컬럼명이 노출되지 않고, 스키마가 바뀌어도 SP만 수정하면 서버 재컴파일이 불필요. 종료 시에는 큐에 남은 작업을 모두 비우는 Drain 루프로 마지막 저장 유실도 막았습니다.

```cpp
// ── 1) 작업 단위 : 요청·응답을 variant 하나로 왕복 ──────────────────
using DBTaskData = std::variant<std::monostate, std::string, player_data>;
struct DBTask {
    int        ioType;      // 완료 시 IOCP에 실어 보낼 io_type
    DBTaskType type;        // LOGIN_AUTH / SAVE_USER_DATA
    int32_t    session_id;
    DBTaskData data;        // 요청=user_id(string) → 결과=player_data 로 제자리 교체
};

// ── 2) 완료 통지 : 별도 채널 없이 IOCP를 재사용 ──────────────────────
DBManager::Instance().Init([](int session_id, int io_type_val, const DBTask& task) {
    EXP_OVER* over = new EXP_OVER;
    over->type    = static_cast<io_type>(io_type_val);
    over->db_task = task;                                   // 결과를 확장 OVERLAPPED에 적재
    PostQueuedCompletionStatus(h_iocp, 1, session_id, &over->over);   // 가짜 I/O 완료 이벤트
});
DBManager::Instance().StartDBThread();

// ── 3) DB 전담 스레드 : 핸들러 맵 디스패치 + 종료 시 Drain ──────────
while (m_db_thread_running) {
    DBTask task;
    if (m_task_queue.try_pop(task)) {                   // PPL lock-free 큐
        auto it = m_handlers.find(task.type);
        if (it != m_handlers.end()) it->second(task);     // ODBC 블로킹은 여기서만
        if (m_notifyCb) m_notifyCb(task.session_id, task.ioType, task);
    }
    else std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
while (m_task_queue.try_pop(task)) { ... }              // 종료 전 잔여 작업 전량 처리

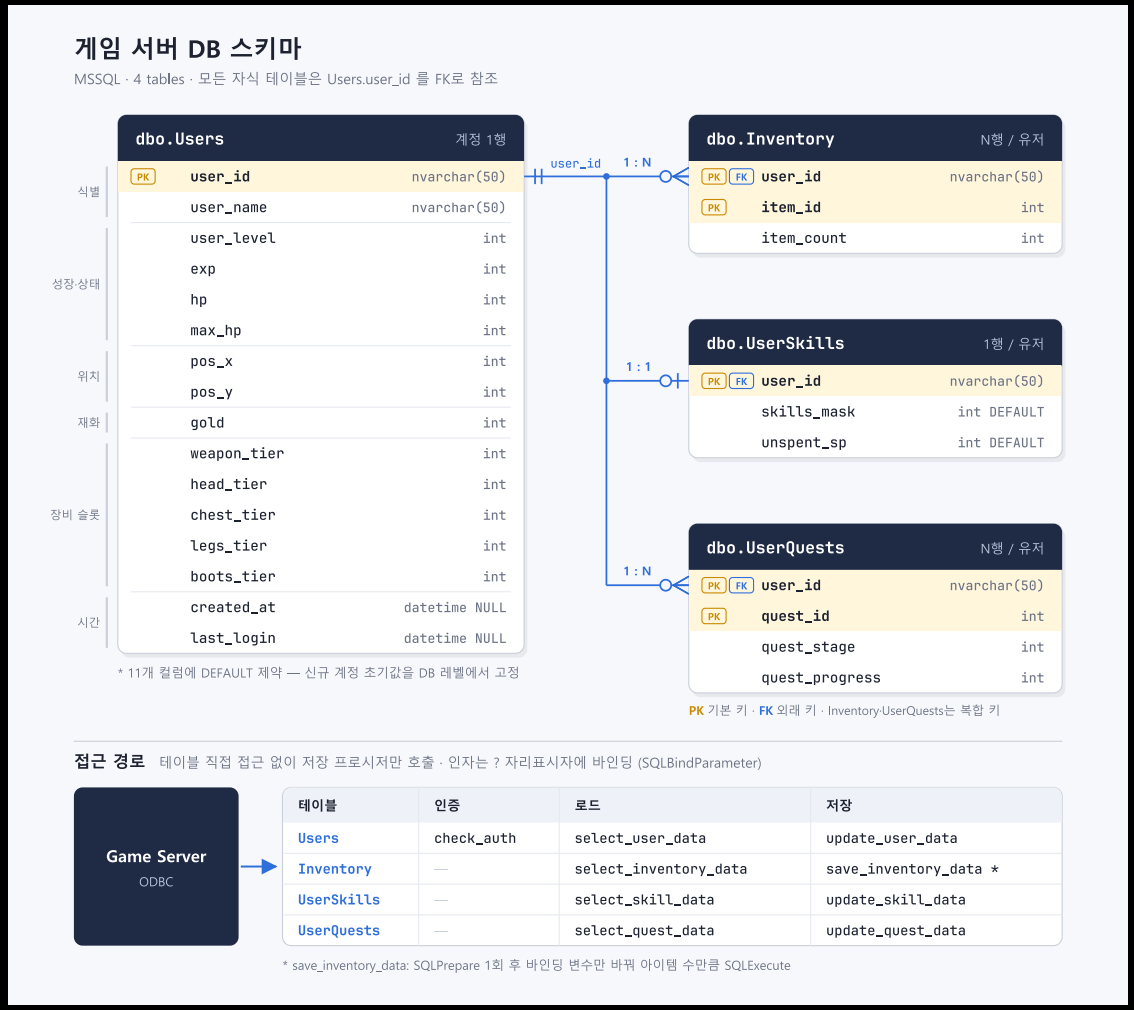
// ── 4) 워커 스레드 : 기존 switch에 case 하나 추가로 끝 ──────────────
case io_type::db_player_auth: {
    player_data result = std::get<player_data>(o->db_task.data);
    if (result.success) new_session->player_->x_ = result.x;   // 로드 데이터 주입 후 입장
}
```

DB 설계

스키마 설계: MSSQL에 계정 1행인 **Users**와 유저당 N행인 자식 테이블 3개(**Inventory · UserSkills · UserQuests**)로 정규화했습니다. 인벤토리·퀘스트는 (user_id, item_id / quest_id) **복합 PK**로 중복 행을 막고, 모든 자식 테이블은 **FK**로 Users를 참조해 고아 데이터가 남지 않게 했습니다. 신규 계정 초기값은 **DEFAULT 제약** 11개로 DB 레벨에서 고정하고, 습득 스킬은 비트마스크 int 1개로 압축해 저장합니다.

접근 계층: 게임 서버는 테이블에 직접 접근하지 않고 **저장 프로시저 9개**(인증 1 · 로드 4 · 저장 4)만 호출합니다. C++ 코드에 테이블·컬럼명이 없어 스키마를 바꿔도 SP만 수정하면 되고, DB 계정에는 EXECUTE 권한만 부여할 수 있습니다.

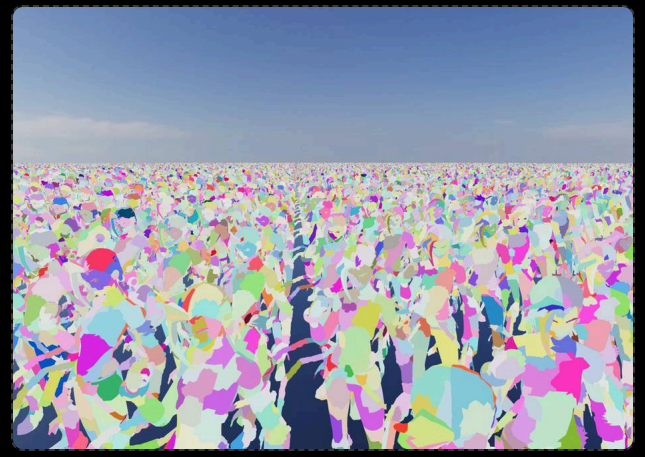
안전한 호출: SP 인자는 문자열로 조립하지 않고 **{CALL sp(?)} + SQLBindParameter**로 바인딩해 SQL 인젝션을 차단했습니다. 인벤토리 저장은 SQLPrepare 1회 후 바인딩 변수만 바꿔 SQLExecute를 반복해 파싱·핸들 할당 비용을 줄였습니다.



Nanite Renderer & Virtual Shadow Map 연구 및 구현

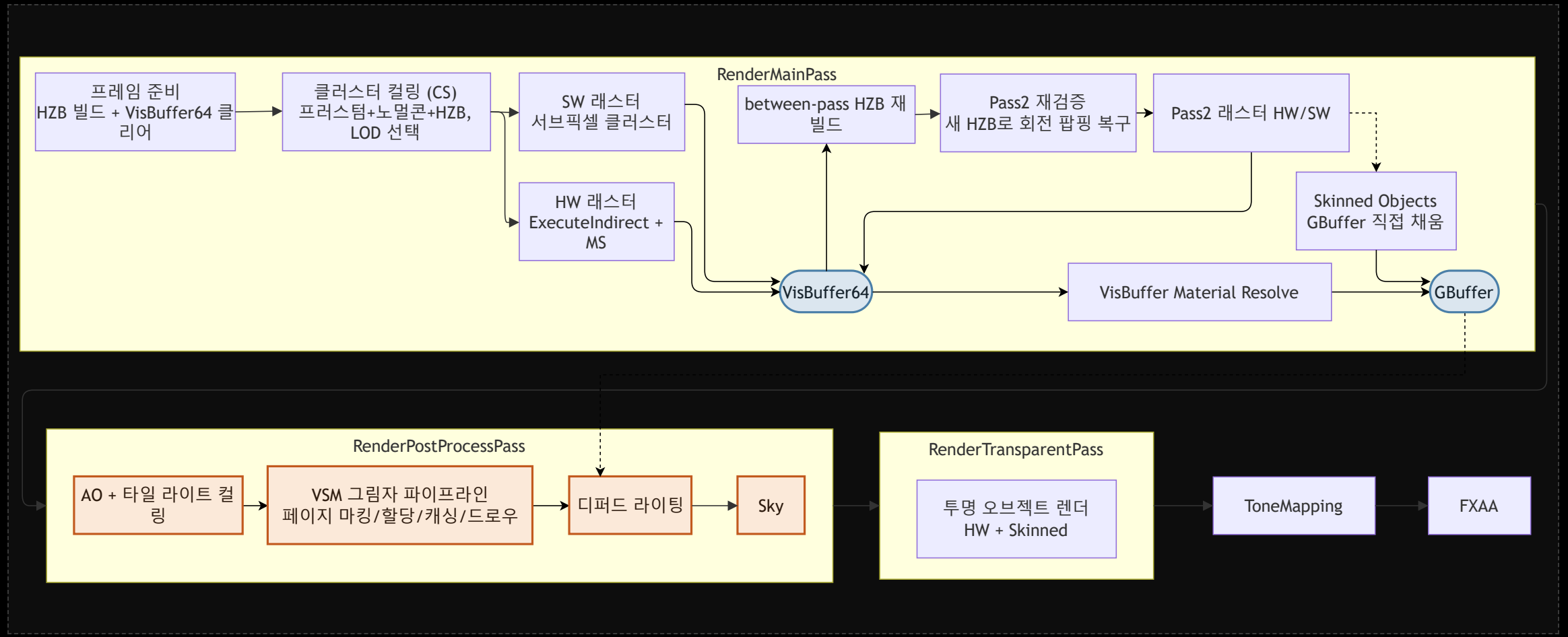
프로젝트 개요





프로젝트 이름	Nanite Renderer & Virtual Shadow Map 연구 및 구현
설명	현장 실습 업체에 존재하던 기존 레거시 형태의 렌더러를 UE엔진 나나이트 기반 논문들을 참고하여 메쉬 셰이더 기반의 현대적 파이프라인 렌더러 제작 및 성능 개선 + 실시간 Local Light Shadow 렌더링 제한 완화 연구 및 구현, 최적화
개발 인원	1명
담당 역할	연구개발, 구현
AI 활용	claude code 를 github mcp와 draw.io mcp를 사용하여 계획 수립 및 실제 UE 조사 skill 생성 및 발전, 코드 생성에 사용함. Antigravity와 claude code를 같이 사용하여 인수인계 및 설명 문서화 작업에 사용
개발 언어	C++, HLSL, DX12
사용 툴 & 라이브러리	Visual Studio, Git
제작 기간	2026 / 07 / 01 ~ 2026 / 08 / 31

렌더러 아키텍쳐


프레임 준비
HZB 빌드 + VisBuffer64 클리어
클러스터 컬링 (CS)
프러스텀+노멀콘+HZB, LOD 선택
SW 래스터
서브픽셀 클러스터
HW 래스터
ExecuteIndirect + MS
RenderMainPass
between-pass HZB 재빌드
Pass2 재검증
새 HZB로 회전 팝핑 복구
Pass2 래스터 HW/SW
Skinned Objects
GBuffer 직접 채움
VisBuffer64
VisBuffer Material Resolve
GBuffer
RenderPostProcessPass
AO + 타일 라이트 컬링
VSM 그림자 파이프라인
페이지 마킹/할당/캐싱/드로우
디퍼드 라이팅
Sky
RenderTransparentPass
투명 오브젝트 렌더
HW + Skinned
ToneMapping
FXAA

클러스터 LOD 계층 구조

Problem: 직지사 씬(삼각형 5,545만 개)에서 서브메시 단위 컬링(레거시)이 병목 - 60fps

Solution: 지오메트리를 클러스터(64정점/128삼각형) 단위로 쪼갠 뒤, 인접 클러스터를 그룹으로 묶어 병합·단순화하는 과정을 반복해서 오프라인에 LOD 계층(DAG)을 미리 구축.
런타임에는 매 프레임 카메라 기준으로 각 클러스터의 화면 투영 오차를 계산해서, 오차가 임계값 밑으로 떨어지는 가장 성긴 레벨을 GPU에서 자동으로 선택 — 화면에서 작게 보이는 부분은 성긴 클러스터로, 크게 보이는 부분은 조밀한 클러스터로 자연스럽게 전환되고, 클러스터 경계 정점을 고정해두는 방식으로 팝핑도 방지

Performance Gain: 레거시 대비 약 3.3배(60→~200fps)

클러스터마다:
    내_오차 = 클러스터_오차 / 카메라까지_거리
    부모_오차 = 부모그룹_오차 / 카메라까지_거리

    if 내_오차 < 임계값 AND 부모오차 >= 임계값:
        이 클러스터를 그린다. #그릴만큼 성기지만, 더 성기면 안되는 딱 그레벨



LOD1



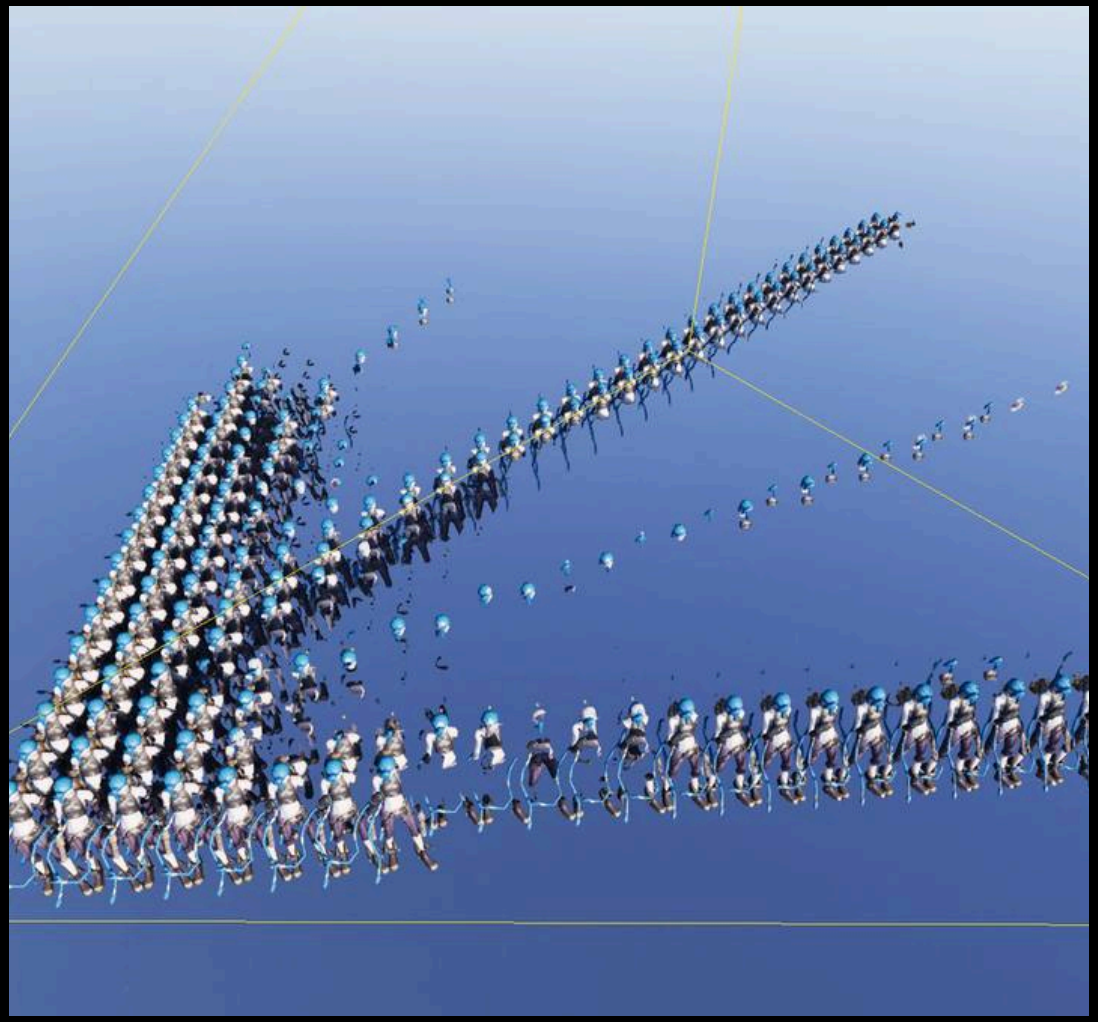
LOD2

GPU 드리븐 2-Pass 컬링

Problem: CPU 기반 가시성 컬링은 1프레임 지연 → 카메라 회전 시 팝핑, 서브메시 단위라 컬링 세밀도 부족

Solution: 컬링을 GPU의 영구 스레드가 전역 카운터에서 작업을 계속 가져가는 워크스틸링 구조로 처리 — 스레드그룹이 노는 시간 없이 그룹(LOD 그룹) 단위로 오차 임계값을 먼저 검사해서, 기준 미달 그룹은 그 안의 클러스터 스레드 자체를 아예 안 띄우고, 통과한 그룹만 클러스터 단위로 프러스텀+노멀콘+HZB 오클루전을 테스트.
오클루전은 2-Pass로 나눠서: Pass1은 지난 프레임의 HZB로 컬링하고 그 결과로 이번 프레임 HZB를 새로 만들고, Pass2는 Pass1에서 "지난 프레임 기준으로만" 가려졌던 후보들을 새 HZB로 다시 검사-팝핑제거

Performance Gain: 컬링 지연 1프레임→0프레임, 기존 LP 큐 구조를 완전히 대체. 인스턴싱 스트레스 테스트(35,000개, 15.6억 삼각형)에서 실제 가시 삼각형은 840만 개 — 컬링/LOD만으로 약 185배 절감,
RTX 3060 12GB 기준 60fps 유지



② GPU 컬링 — 워크스틸링 + 2단계 오클루전

```
# 여러 스레드가 동시에, 전역 카운터에서 계속 다음 그룹을 가져감
반복:
    그룹 = 다음_작업_가져오기()
    if 그룹.오차가_너무_큼: continue   # 그룹 하나가 안에 있는 클러스터 전부를 대표해서 스킵

    for 클러스터 in 그룹:
        if 화면_밖 or 뒤쪽_면: continue

        if 지난_프레임_깊이맵으로_가려짐_판정():
            "혹시 가려진 후보" 목록에 저장       # 1단계
        else:
            그린다

# 깊이맵을 새로 만든 뒤 — 후보였던 것만 다시 검사
for 클러스터 in 혹시_가려진_후보:
    if 이번_프레임_깊이맵으로는_안_가려짐:
        그린다      # 카메라가 돌아서 새로 드러난 것들, 2단계
```

HW/SW 하이브리드 래스터라이저 + Visibility Buffer

Problem: 하드웨어 래스터라이저는 2×2 쿼드 단위 강제 실행이라, 화면에서 1픽셀보다 작은 서브픽셀 삼각형에서 최대 4배 낭비(quad overdraw) — 클러스터로 삼각형 수를 줄여도 안 풀리는 별개 축의 비용

Solution: 화면 투영 크기가 작은 클러스터만 컴퓨트 셰이더 소프트웨어 래스터로 우회, 64비트 원자적 버퍼(VisBuffer64) 하나에 HW/SW 양쪽이 깊이+ID를 같이 기록해 지연 셰이딩과 통합

Performance Gain: 위 인스턴싱 테스트에서 SW 래스터 OFF 60fps → ON 70fps — 서브픽셀 클러스터가 실제로 발생하는 규모에서 quad overdraw 제거가 순이득으로 확인됨



[빨간색으로 덮인 부분이 SW레스터라이저가 적용되는 삼각형이 있는 메쉿랫 영역]

③ HW/SW 래스터 분기 → 하나의 버퍼로 합류

```
클러스터마다:
    화면에서_보이는_크기 = 클러스터_크기 / 카메라까지_거리

    if 화면에서_보이는_크기 < 1픽셀:
        소프트웨어로_직접_픽셀_채우기()   # 하드웨어처럼 2x2 묶음으로 낭비 안 함
    else:
        하드웨어_래스터라이저_사용()

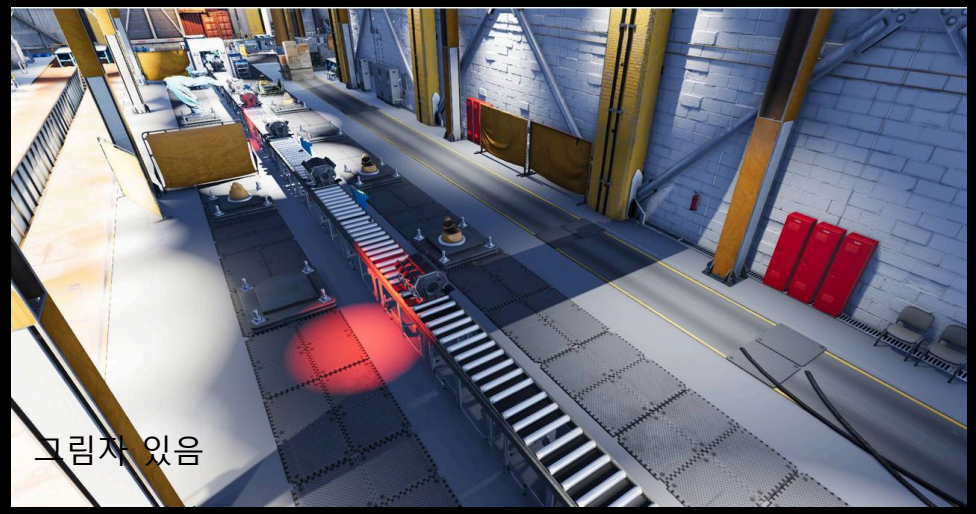
    # 둘 다 결과는 같은 곳에: 깊이+ID를 원자적으로 기록
    버퍼[픽셀] = 더_가까운_값이면_덮어쓰기(깊이, 클러스터ID)
```

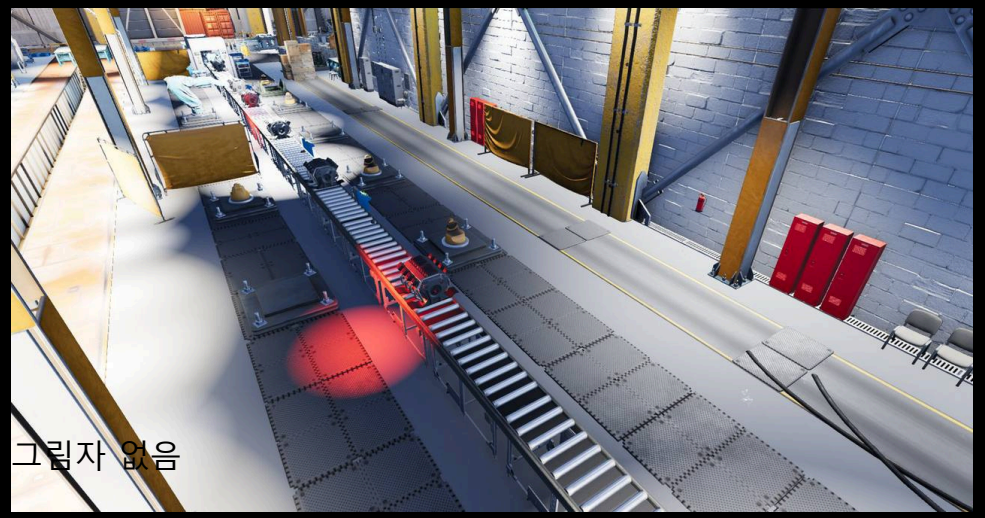
Virtual Shadow Map — 다중 포인트/스팟 라이트 그림자

Problem: 포인트 라이트 하나 = 큐브맵 6면(씬 6번 그리기), 라이트가 늘수록 비용 선형 폭증

Solution: 그림자맵을 128×128 페이지 단위로 가상화, GPU 드리븐 라이트 선정 + 페이지 캐싱 + 점유율 기반 자동 해상도 조절

Performance Gain: 점조명 400개, 소프트 조명 400개 환경 씬 기준 캐싱으로 3.8배 성능 이득(50→190fps),
그림자 맵을 페이지 단위로 가상화하여 물리 VRAM 최적화: 지역조명 그림자 맵의 갯수에 비례하던 VRAM사용량을 (점조명 512*512, 스포트 1024*1024) -> 4096(가로) × 8192(세로) = 33,554,432픽셀 = 134MB로 고정





SIDE PROJECTS


| PROJECT


| PM GAME


| 메탈슬러그 모작

게임 엔진 프로젝트 unity 개발 프로젝트

- unity 기능을 최대한 활용한 게임 개발
- 스플라인, 프리팹, 콜리전, 콜라이더, Scene Hirachy

PM GAME 컴퓨터 그래픽스

- 컴퓨터 그래픽스 및 OpenGL 사용한 게임개발
- 고급 텍스쳐링 기법 사용, 직접광, 스포트 라이트, 점 조명 구현
- 씬, 메인루프, 입력, 오브젝트 생성 삭제, 그래픽스 엔진, glsl, 기본적인 3D 자체 게임 엔진

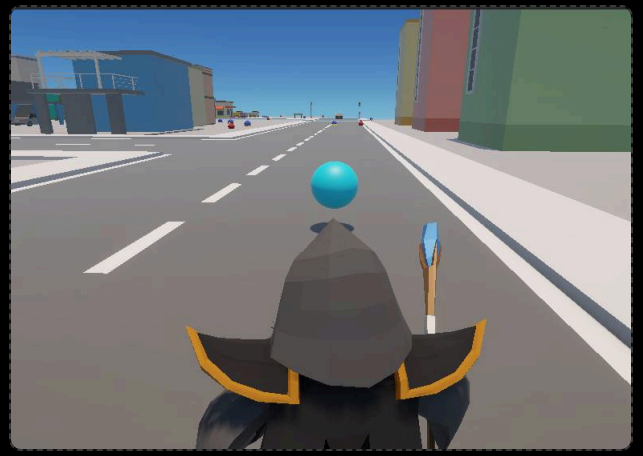
메탈 슬러그 모작 windows programming

- win32사용한 2D 게임 개발
- 충돌체와 이동 속도 계산을 위한 정밀한 물리 도입
- 2D 애니메이션을 위한 세밀한 상태머신 개발



Unity Project

프로젝트 개요



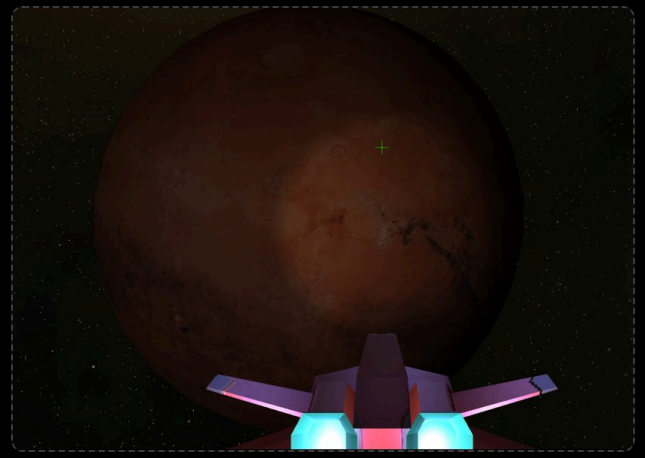


프로젝트 이름	Unity Project
장르	TPS,레이싱 등
설명	유니티 6.0을 사용한 여러 프로젝트 (5개)입니다
개발 인원	1명
담당 역할	클라이언트 개발
개발 언어	untiy6, c#
사용 툴 & 라이브러리	Visual Studio, Git, unity
제작 기간	2025 / 05 ~ 2025 / 06 (한달)

PM GAME | 3D Space Shooting Game

프로젝트 개요





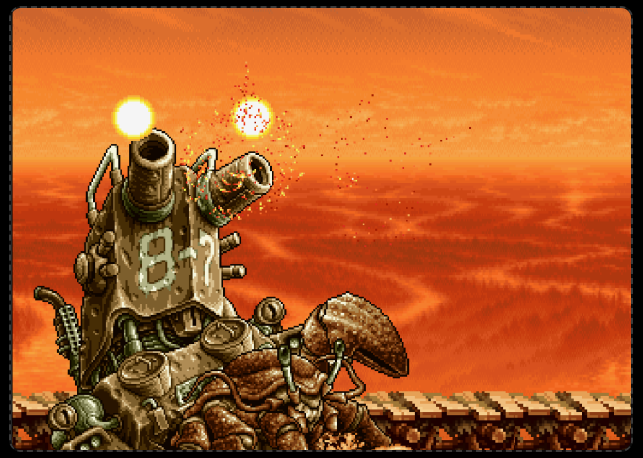
프로젝트 이름	PM GAME
장르	3D 1/3인칭 슈팅게임
설명	3D 1/3인칭 우주선 슈팅 게임, 날아오는 운석을 파괴하라
개발 인원	2명
담당 역할	그래픽스, 오브젝트 관리, 게임 메인 루프, 입력, 사운드, 조명, 적 로직, 플레이어 로직 등 거의 모든 클라이언트 개발 및 자체 엔진 개발 제작
개발 언어	C++20
사용 툴 & 라이브러리	Visual Studio, Git, OpenGL, GLM, fmod
제작 기간	2024 / 10 ~ 2024 / 11



Metal Slug 모작

프로젝트 개요





프로젝트 이름	메탈슬러그 모작
장르	2D 플랫포머 슈팅
설명	메탈슬러그 게임을 win32라이브러리를 사용해서 모작을 만들었습니다.
개발 인원	2명
담당 역할	애니메이션 스프라이트 편집, 상태머신 작성, 플레이어 로직, 보스와 적의 공격 로직 모든 리소스 관리,편집
기술 스택	C++20, win32, fmod
사용 툴 & 라이브러리	Visual Studio, Git
제작 기간	2024 / 05 ~ 2024 / 06



Thank You

포트폴리오를 검토해 주셔서 감사합니다.

Author : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738